

Sign Language Detection System

Recognising Five Hand Signs in Real Time — Preprocessing, Feature Engineering and Model Selection

Machine Learning Project Report

Baseline CNN vs. MediaPipe Landmark Model · Built with Streamlit

Task 3

Contents

1. Problem Statement
2. Data Collection — Why a Custom Dataset Was Needed
3. Data Preprocessing
4. Feature Engineering
5. Model Selection and Comparison (Baseline vs. Advanced)
6. Why the Camera Could Not Focus on the Hand
7. Visualizations and Insights
8. The Streamlit GUI and Time Restriction
9. Results Summary
10. Limitations
11. Conclusion and Future Work

1. Problem Statement

The task is to build a machine learning system that recognises sign language for a small set of chosen words, wired into a graphical interface that supports both image upload and a live webcam feed. The system must also be operational only within a fixed daily window — here, 6 PM to 10 PM. Evaluation is based on the overall behaviour of the system and the interface rather than raw accuracy alone, which makes the data and engineering decisions as important as the model itself.

Five signs were chosen — Hello, Yes, No, I Love You and Water — each selected to have a clearly different hand shape so a model can tell them apart. This report walks through the data collection, preprocessing, feature engineering and model selection, and compares a baseline image model against the final landmark-based model that the application uses.

2. Data Collection — Why a Custom Dataset Was Needed

The first instinct was to use an existing public sign-language dataset, but none fit a word-level image classifier for the five chosen signs, and confirming this took real effort.

- The popular ASL datasets on Kaggle contain only the alphabet — one folder per letter, A to Z. They have no whole-word signs such as Hello or Water, so they cannot teach the words the task asks for.
- The few word-level datasets that do exist were the wrong shape: large collections such as WLASL are videos rather than still images, and others were supplied as object-detection data with bounding boxes (YOLO format) rather than simple class folders, which did not suit a straightforward image classifier and was not permitted for this work.
- Research datasets that did include word gestures kept those words only as videos, while their still-image portion was again just the alphabet.

Because clean, image-classification data for specific whole-word signs essentially does not exist off the shelf, and the brief allows “words of your choice,” the dataset was built first-hand with a webcam. A capture script showed each target sign on screen and recorded samples, and the data was collected across several rounds with the background, lighting and position deliberately changed between rounds. Because every sign was recorded in every environment, no single background could be tied to one sign — forcing any model to learn the hand rather than the room.

```
WORDS = ['Hello', 'Yes', 'No', 'ILoveYou', 'Water']
cap = cv2.VideoCapture(0)
for word in WORDS:
    save_dir = os.path.join('data/train', word)
    os.makedirs(save_dir, exist_ok=True)
    count = len(os.listdir(save_dir)) # continue numbering across rounds
    while capturing:
        ok, frame = cap.read()
        frame = cv2.flip(frame, 1)
        cv2.imwrite(f'{save_dir}/img_{count:04d}.jpg', frame)
        count += 1
```

[INSERT IMAGE: the capture window — webcam view with the on-screen sign name and live counter.]

Around 400 samples were captured per sign, giving a balanced dataset of roughly 2,000 images across the five classes.

3. Data Preprocessing

Two different preprocessing pipelines were used, one for each modelling approach. Keeping them separate and correct mattered, because feeding a model inputs prepared differently from how it was trained silently produces wrong predictions.

3.1 Image Preprocessing (Baseline CNN)

For the image model, every frame was resized to 64×64 pixels, kept in three colour channels (RGB), and scaled so pixel values fell in the 0–1 range. The dataset was loaded with a shuffled split so that training and validation each contained a fair mix of all five signs and all backgrounds. Light augmentation — small rotations, zooms and translations — was applied to the training images only, never to validation.

```
train_ds = tf.keras.utils.image_dataset_from_directory(
    DATA_DIR, validation_split=0.2, subset='training',
    seed=42, image_size=(64,64), batch_size=32) # shuffles before split
normalize = layers.Rescaling(1./255)           # pixels -> 0..1
augment    = tf.keras.Sequential([
    layers.RandomRotation(0.05), layers.RandomZoom(0.1),
    layers.RandomTranslation(0.1, 0.1)])        # training only
```

3.2 Landmark Preprocessing (Advanced Model)

For the final model, preprocessing means running MediaPipe on each frame to detect the hand and return the coordinates of its 21 joints. No pixels are kept — only the joint positions move forward. If MediaPipe finds no hand, the frame is skipped rather than guessed at.

```
res = hands.process(rgb_frame)
if res.multi_hand_landmarks:
    h1 = res.multi_hand_landmarks[0]
    coords = [(lm.x, lm.y) for lm in h1.landmark] # 21 (x, y) joints
```

4. Feature Engineering

This is where the two approaches differ most, and where the project's key idea lives. The image model performs almost no feature engineering: the raw pixels are handed to the network, which is left to discover useful features on its own — and, as Section 5 shows, it discovered the wrong ones.

The landmark model, by contrast, engineers a compact and meaningful feature vector by hand. The 21 joint coordinates are normalised in two steps so that the features describe only the shape of the hand, independent of where it is in the frame or how big it appears:

- Translation invariance — the wrist joint is subtracted from every point, moving the hand to a common origin so its position in the frame no longer matters.

- Scale invariance — the coordinates are divided by the hand's overall size, so the distance from the camera no longer matters.

The result is a 42-value vector (21 joints $\times$ x and y) that captures the finger configuration and nothing else. This single engineering step is the reason the final model generalises to a live camera.

```
def normalize(landmarks):
    pts = np.array(landmarks, dtype=np.float32) # 21 (x, y) points
    pts = pts - pts[0] # 1) wrist to origin (translation)
    m = np.max(np.linalg.norm(pts, axis=1))
    if m > 0:
        pts = pts / m # 2) scale by hand size (scale)
    return pts.flatten() # 42 shape-only features
```



5. Model Selection and Comparison (Baseline vs. Advanced)

5.1 Baseline — Convolutional Neural Network on Images

A Convolutional Neural Network was the natural baseline, because CNNs are the standard tool for image classification: they slide small filters across an image to detect edges, then shapes, then whole hand poses. The network used three convolutional blocks feeding a dense classifier with five softmax outputs, trained on Kaggle with a GPU.

```
model = models.Sequential([
    layers.Conv2D(32,(3,3),activation='relu',padding='same',input_shape=(64,64,3)),
    layers.BatchNormalization(), layers.MaxPooling2D(2,2), layers.Dropout(0.25),
    layers.Conv2D(64,(3,3),activation='relu',padding='same'),
    layers.BatchNormalization(), layers.MaxPooling2D(2,2), layers.Dropout(0.25),
    layers.Conv2D(128,(3,3),activation='relu',padding='same'),
    layers.BatchNormalization(), layers.MaxPooling2D(2,2), layers.Dropout(0.3),
    layers.Flatten(),
    layers.Dense(256,activation='relu'), layers.Dropout(0.5),
    layers.Dense(5,activation='softmax')])
```

On its validation set the CNN reached around ninety percent accuracy and, once an early data-split bug was fixed, spread its predictions across all five signs. On paper it looked finished — yet in the live application it collapsed, predicting a single sign for almost everything. The cause was that consecutive captured frames are nearly identical, so near-duplicates sat on both sides of the train/validation split and inflated the score; the model had memorised the recording sessions and their backgrounds rather than the signs. A live frame, with different lighting and a fresh background, looked unfamiliar, so the model defaulted to one class.

5.2 Advanced — MediaPipe Landmarks with an MLP

Because the failure came from the representation rather than the architecture, the advanced model changes what the model looks at. MediaPipe extracts the hand's 21 joints, the normalisation of Section 4 turns them into 42 shape-only features, and a small Multi-Layer Perceptron (a fully-connected neural network) classifies those features into the five signs. A CNN is deliberately not used here — convolutions exist to scan pixels, and there are no pixels, only a short list of numbers.

```
from sklearn.neural_network import MLPClassifier
clf = MLPClassifier(hidden_layer_sizes=(128, 64),
                    max_iter=600, random_state=42)
clf.fit(X_train, y_train) # X = 42 landmark features per sample
```

Training runs locally in seconds with no GPU, since the input is a table of coordinates. The model reached roughly ninety percent on its held-out test set — but unlike the CNN, this figure held up live, because background and lighting never reach the model.

5.3 Side-by-Side Comparison

Aspect	Baseline — CNN on images	Advanced — MLP on landmarks
Input	64×64 RGB pixels	42 normalised landmark values
Feature engineering	None (learned from pixels)	Translation + scale-invariant geometry
Framework	TensorFlow / Keras	MediaPipe + scikit-learn
Hardware	GPU (Kaggle)	Laptop CPU, trains in seconds
Validation accuracy	≈ 90%	≈ 90%
Live webcam	Collapsed to one class	Works reliably
Why	Memorised background	Sees only hand shape

The two models score almost the same on validation, but only the landmark model survives live use — a reminder that validation accuracy alone can be misleading when the data contains near-duplicates, and that the right representation matters more than the raw number.

6. Why the Camera Could Not Focus on the Hand

A core difficulty with the image approach is that a webcam does not see a hand — it sees a whole scene. A frame contains the face, the upper body, the background and the lighting of the room, with the hand occupying only a small part of it. The CNN was given that entire frame and left to work out which part mattered, and with limited, self-recorded data it latched onto the easiest signal available: the background and overall appearance of each recording session. Nothing in the pipeline told it to concentrate on the hand.

MediaPipe solves exactly this problem. It is a dedicated hand-tracking model that locates the hand within the scene and returns only its joint positions, discarding everything else. In effect it does the focusing that the camera and the CNN could not: it isolates the hand and hands the model a clean description of it. This is the single biggest reason the project moved from a plain CNN on full frames to MediaPipe landmarks — the hand is found explicitly, rather than hoped for implicitly.

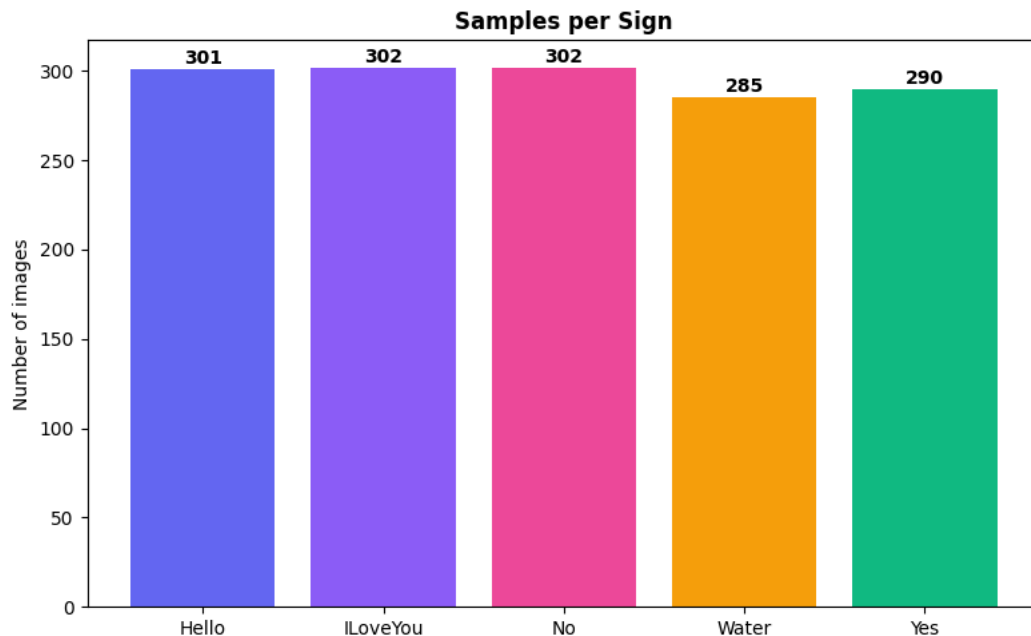
7. Visualizations and Insights

Several visualizations were produced with Matplotlib and Seaborn to understand the data and evaluate the models. The code used to generate each is shown alongside the figure it produces.

7.1 Class Balance

A bar chart of samples per sign confirms the dataset is balanced, with roughly 400 images each. Balance matters because an imbalanced set would let a model score well simply by favouring the largest class.

```
counts = {d: len(os.listdir(os.path.join(DATA_DIR,d)))  
          for d in sorted(os.listdir(DATA_DIR))}  
plt.bar(counts.keys(), counts.values())  
plt.title('Samples per Sign'); plt.ylabel('Images')  
plt.savefig('samples_per_sign.png', dpi=150)
```



7.2 Example of Each Sign

A grid showing one captured frame per sign documents the five distinct hand shapes and the varied backgrounds used during capture, making the diversity of the dataset visible at a glance.



7.3 Per-Class Accuracy

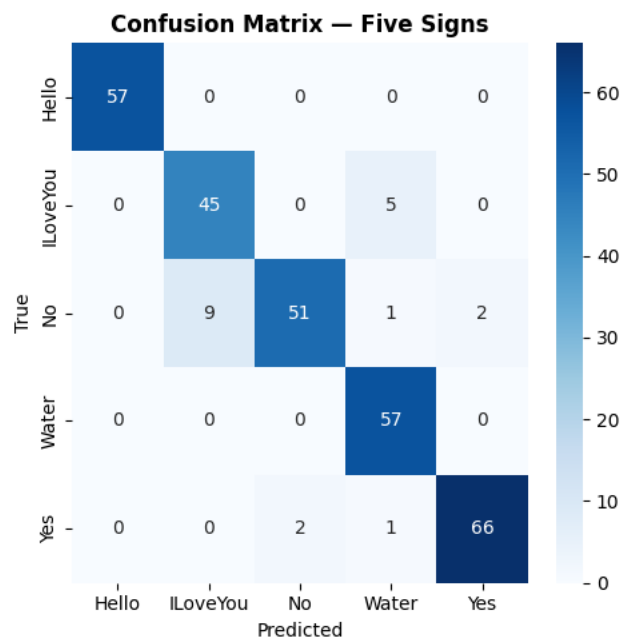
A bar chart of per-class accuracy shows how well each sign is recognised. The insight here is consistent across runs: the visually distinct signs (Hello, Water, I Love You) score highest, while the two most compact hand shapes (Yes and No) are the hardest, since a fist and a two-finger pinch look similar at low resolution.

```
accs = [ (all_pred[all_true==i]==i).mean()*100
         for i in range(len(class_names)) ]
plt.bar(class_names, accs)
plt.ylabel('Accuracy (%)'); plt.title('Per-Class Accuracy')
plt.savefig('per_class_accuracy.png', dpi=150)
```

7.4 Confusion Matrix

A confusion matrix heatmap shows exactly where mistakes happen. A bright diagonal indicates mostly-correct predictions, and the main off-diagonal cell — between Yes and No — confirms the expected confusion between the two most similar hand shapes.

```
import seaborn as sns
from sklearn.metrics import confusion_matrix
cm = confusion_matrix(all_true, all_pred)
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=class_names, yticklabels=class_names)
plt.title('Confusion Matrix'); plt.savefig('confusion_matrix.png', dpi=150)
```



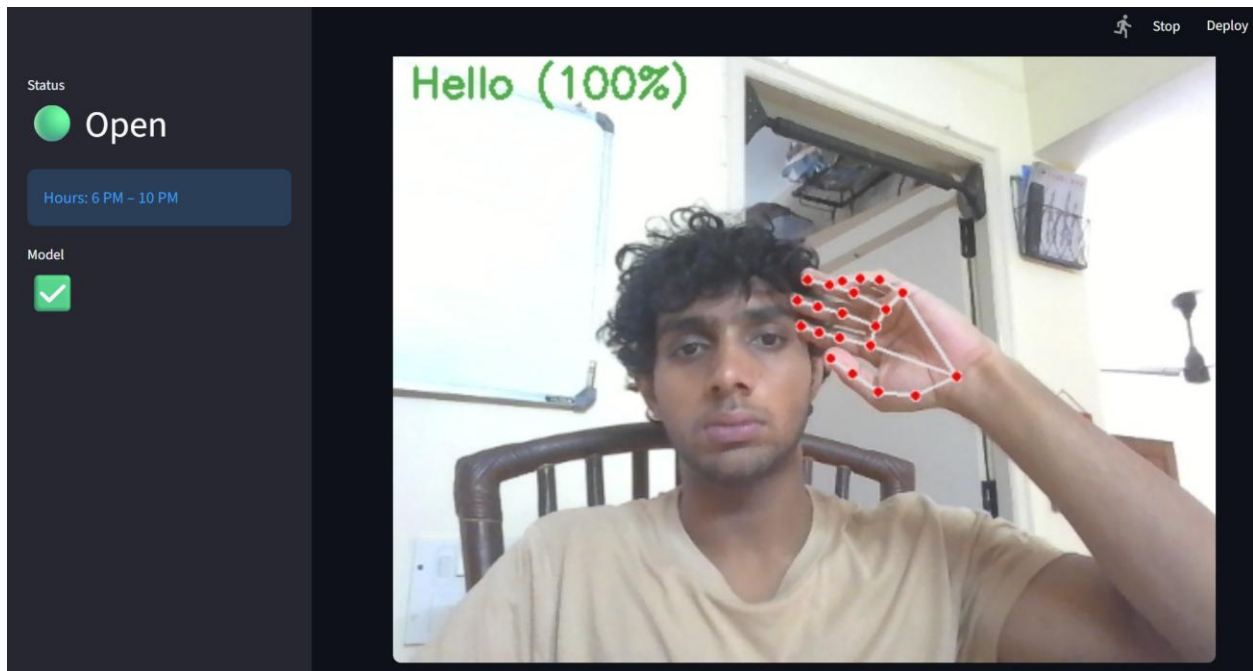
8. The Streamlit GUI and Time Restriction

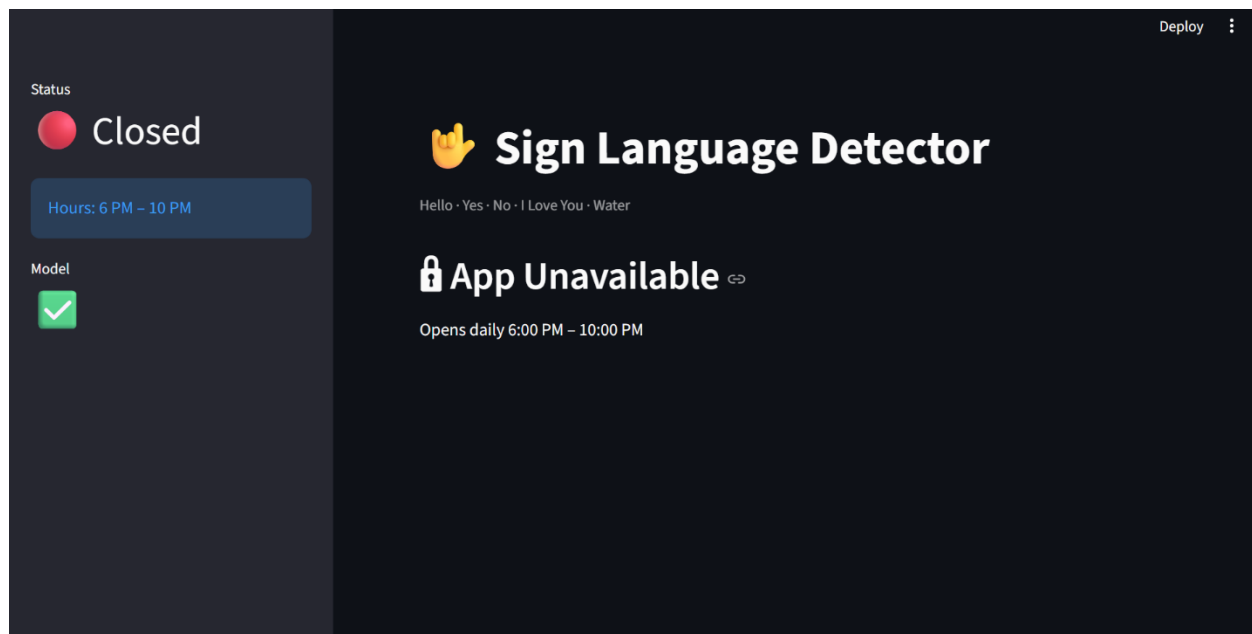
The interface was built with Streamlit, which turns a Python script into a clean web app with little extra code. It offers two modes. In upload mode, a photo is processed, the hand skeleton is

drawn on it, and the predicted sign, confidence and a probability bar chart are shown. In live mode, a toggle opens the webcam and the detect → normalise → classify loop runs on each frame, drawing the skeleton and prediction onto the video; predictions are smoothed over a short rolling window so a single noisy frame cannot make the label flicker.

The 6–10 PM restriction is enforced by a small time-check that runs before any detection feature is shown. Inside the window the full interface is unlocked; outside it, the tools are replaced by a lock screen stating when the app reopens.

```
START_HOUR, END_HOUR = 18, 22 # 6 PM to 10 PM
def time_open():
    return START_HOUR <= datetime.now().hour < END_HOUR
```





9. Results Summary

Stage	Choice	Result
Data collection	Custom webcam capture, multi-background	Balanced ~400 samples per sign
Preprocessing	Landmark extraction via MediaPipe	Hand isolated from the scene
Feature engineering	Translation + scale normalisation	42 shape-only features
Baseline model	CNN on images	≈ 90% validation, failed live
Final model	MLP on landmarks	≈ 90% test, works live
Interface	Streamlit, upload + live, 6–10 PM gate	All requirements met

The decisive outcome is qualitative as much as numerical: where the baseline CNN collapsed to a single prediction on the live webcam, the landmark model recognises all five signs in real time under everyday conditions, with Yes and No the only pair that occasionally trades places.

10. Limitations

While the final system meets the task requirements and works reliably in everyday conditions, it has a few honest limitations worth recording.

10.1 Trained on a Single Person

All the data was captured from one person's hands in a handful of sessions. The model therefore learns that individual's hand proportions and signing style, and may be less accurate for other people whose hands differ in size or shape. A genuinely robust system would need data from many different signers.

10.2 A Small Vocabulary of Static Signs

Only five signs are supported, and each is treated as a single static hand pose. Real sign language is far larger and many signs involve motion, which a one-frame-at-a-time classifier cannot capture — it sees only the key pose, not the movement, which limits how many real signs this approach can distinguish.

10.3 Dependence on Hand Detection

The whole pipeline relies on MediaPipe finding the hand. If the hand is poorly lit, partially out of frame, turned side-on, or moving quickly, MediaPipe may fail to detect it, and with no landmarks the model cannot predict at all. The system is only as reliable as the hand detection beneath it.

11. Conclusion and Future Work

This project shows that the hardest part of a sign-recognition system is often the data and the representation, not the network. A custom dataset had to be built because no off-the-shelf data matched the chosen words. A baseline CNN trained on the raw images appeared to work, with around ninety percent validation accuracy, yet failed on a live camera because it memorised backgrounds and near-duplicate frames rather than the signs, and because nothing forced it to focus on the hand within a busy scene.

The working system replaces pixels with geometry: MediaPipe locates the hand and extracts 21 joints, a normalisation step removes position and scale so only the hand shape remains, and a small neural network classifies that shape. Because background and lighting never reach the model, live performance finally matches training. A time-check enforces the 6–10 PM rule, and a Streamlit interface delivers both image-upload and real-time video modes.

Future improvements would include expanding the vocabulary, collecting data from several different people so the model recognises anyone's hands, incorporating both hands and motion to support signs that depend on movement, and adding a confidence-based “no sign detected” state so the system stays silent until it is genuinely sure.